Synapse: A Constraint-Based Matchmaking Platform for Hackathon Team Formation

Raghul M and Rithesh Madhav S
Rajalakshmi Engineering College

Abstract
Hackathon team formation is commonly supported by free-text directories in which participants describe skills, roles, experience, and preferences through unconstrained language. Equivalent concepts may be entered inconsistently, increasing normalization cost, lowering match precision, and forcing manual comparison. This design paper specifies Synapse, a constraint-based matchmaking platform that models human-capital allocation as a Constraint Satisfaction Problem. Critical matching attributes are represented through finite database domains, including enumerated project roles and experience levels, while technology stacks are retained as structured sets. A two-phase pipeline is proposed. Hard constraints first eliminate infeasible candidates through role parity, minimum experience, availability, and environment compatibility. Feasible candidates are then ranked through Jaccard technology-stack similarity and weighted contextual preferences. The architecture combines Next.js 14 App Router, React Server Components, PostgreSQL through Supabase and PgBouncer, Prisma ORM, NextAuth.js v5, and Vercel serverless deployment. The design supports constant-time categorical verification, linear candidate filtration, and explainable match decisions. This paper presents the specified system architecture and algorithmic model; it distinguishes the proposed CSP engine from any separate implementation-verification exercise.

1. Introduction

Hackathons impose an unusual team-formation problem. Participants must identify complementary abilities, assess constraints, and organize collaboration within a limited time window. Conventional directories provide profile lists, but a free-text representation does not establish a stable computational basis for matching. A user may describe a frontend capability as React, ReactJS, React.js, UI development, or web engineering. Similar ambiguity occurs for roles, experience, location, event mode, and technology preferences.

If N profiles are repeatedly compared through unnormalized text attributes, manual or pairwise comparison can approach O(N^2) work. Synapse instead defines a finite candidate domain and evaluates explicit predicates. The resulting design separates feasibility from desirability: a candidate must first satisfy absolute constraints, then may be ranked among feasible alternatives.

2. Contributions

The proposed system contributes: (i) finite-domain modeling for roles and experience, (ii) deterministic hard-constraint filtering, (iii) Jaccard-based technology scoring, (iv) environment-aware ranking, and (v) a serverless, type-safe architecture that supports explainable decisions.

3. System Architecture

Synapse follows a decoupled full-stack topology. Next.js 14 supplies App Router routing, layouts, React Server Components, and trusted route handlers. Interactive controls such as filters and forms are implemented as client components, while data retrieval and protected matching operations are placed in server-side contexts. Prisma constructs type-safe relational queries. Supabase provides PostgreSQL persistence, and PgBouncer on port 6543 pools database connections for short-lived Vercel functions.

Browser → Next.js Route or Server Action → Session Verification → Prisma → PgBouncer:6543 → Supabase PostgreSQL

Vercel distributes static assets through its edge network and executes dynamic application logic in serverless functions. This separation avoids exposing database credentials to browser clients and permits authentication checks before profile, application, favorite, and team-requirement mutations are executed.

4. Data Modeling and Finite Domains

Algorithmic matching requires comparable data. A finite domain prevents semantically equivalent values from being treated as unrelated text. ProjectRole may include FRONTEND_ENGINEER, BACKEND_ENGINEER, FULL_STACK_ENGINEER, AI_ML_ENGINEER, UI_UX_DESIGNER, DEVOPS_ENGINEER, BLOCKCHAIN_ENGINEER, and PRODUCT_MANAGER. ExperienceLevel may include BEGINNER, INTERMEDIATE, ADVANCED, and EXPERT.

Let the experience domain be E = {BEGINNER, INTERMEDIATE, ADVANCED, EXPERT}. An ordinal mapping phi assigns the values 1 through 4. The required experience predicate is therefore evaluated without natural-language interpretation.

$$\text{ExperienceMatch}(u,t) = 1 \text{ if } \varphi(u.\text{experience}) \geq \varphi(t.\text{requiredExperience}); \text{ otherwise } 0$$

The central entities are User, TeamRequirement, Application, Skill, Favorite, Account, and Session. A User may own team requirements and applications. An Application connects one candidate with one requirement, and a uniqueness rule UNIQUE(candidateId, teamRequirementId) prevents duplicate applications. A TeamRequirement stores the required role, experience floor, structured technology set, event mode, location, availability information, and open positions.

5. Security and Implementation

NextAuth.js v5 enforces zero-trust access using server-validated sessions and OAuth 2.0 providers. A JWT represents authenticated state, but authorization for sensitive operations must be derived from the verified server session rather than from a client-submitted user identifier. This boundary is essential for profile mutation, application creation, and favorite management. Deployment must use locked dependencies, generated Prisma artifacts, verified environment variables, and a PgBouncer-compatible database URL.

6. Matchmaking Algorithm as a CSP

A Constraint Satisfaction Problem is represented as P = (X, D, C), where X denotes variables, D denotes finite domains, and C denotes constraints. For an open team position, the candidate variable x is selected from the domain of user profiles. Each candidate is evaluated against role, experience, availability, and environmental constraints.

$$F = \{u \in U \mid C_{role}(u,t) \land C_{experience}(u,t) \land C_{environment}(u,t) \land C_{availability}(u,t)\}$$

6.1 Phase 1: Hard Constraints

Hard constraints are non-negotiable. Role parity is evaluated by direct enum equality. Experience compatibility is evaluated through the ordered finite experience domain. A candidate failing any predicate is excluded before similarity ranking occurs.

$$C_{role}(u,t) = 1 \text{ if } u.role = t.requiredRole; \text{ otherwise } 0$$

$$H(u,t) = C_{role}(u,t)\ \ C_{experience}(u,t)\ \ C_{environment}(u,t)\ \ C_{availability}(u,t)$$

Each candidate is evaluated once. Since role and experience checks are constant-time predicates, full hard filtering across N candidate profiles is O(N). The procedure is deterministic: identical data and constraints produce the same feasible set. A rejection can be explained through an explicit failed predicate rather than an opaque score.

6.2 Phase 2: Soft Constraints

Feasible candidates are ranked using structured technology stacks. Let S_u denote a candidate stack and S_t the required stack. Jaccard similarity rewards overlap while penalizing excess unrelated technologies only through the union term.

$$J(S_u,S_t) = |S_u \cap S_t| \,/\, |S_u \cup S_t|$$

Event mode and location may be represented by an environment weight. For online events, remote compatibility is assigned full weight. For offline events, a candidate must satisfy a location requirement. Hybrid events may permit remote participation when explicitly authorized.

$$Score(u,t) = \alpha J(S_u,S_t) + \beta W_{environment}(u,t) + \gamma W_{availability}(u,t) + \delta W_{preference}(u,t)$$

The parameters satisfy $\alpha + \beta + \gamma + \delta = 1$. Technology overlap may receive the highest weight, while environmental compatibility, schedule compatibility, and optional preferences refine the ranking. If K candidates survive hard filtering and m is the bounded average stack size, scoring is $O(K \cdot m)$ and ordering is $O(K \log K)$.

7. Discussion

Finite domains reduce ambiguity and permit O(1) categorical verification. The complete hard-filter pass remains O(N), avoiding unnecessary comparison of users who cannot satisfy mandatory requirements. The two-phase pipeline preserves an important distinction: high technology similarity cannot compensate for a missing mandatory role or insufficient experience. This improves transparency for both organizers and candidates.

The model has limitations. Enumerations improve consistency but can be rigid when new specializations emerge. Controlled schema migrations are required to introduce new roles without creating uncontrolled synonyms. Jaccard similarity also treats skills uniformly unless domain-specific weights are introduced. Finally, self-reported skills do not constitute independently verified competence.

8. Conclusion and Future Work

Synapse specifies an explainable foundation for hackathon matchmaking through finite-domain data modeling, deterministic eligibility filtering, and heuristic ranking. Enumerated roles and experience levels turn ambiguous input into comparable state, while Jaccard similarity ranks feasible candidates by technology overlap. The proposed architecture combines Next.js, React Server Components, Supabase PostgreSQL, PgBouncer, Prisma, NextAuth.js, and Vercel to support secure, scalable execution.

Future work should introduce natural-language classification during onboarding, with user confirmation before a free-text description is mapped to a finite role or skill category. GitHub API integration may provide live public metrics such as repository activity, language distributions, and project history. The next algorithmic extension is multi-member combinatorial optimization, in which a team is selected jointly to maximize role coverage, skill diversity, availability overlap, and contextual compatibility under explicit constraints.

References

[1] R. Dechter, Constraint Processing. Morgan Kaufmann, 2003.

[2] T. Lappas, K. Liu, and E. Terzi, “Finding a team of experts in social networks,” Proceedings of KDD, 2009.

[3] Next.js, “Next.js 14 App Router Documentation,” 2024.

[4] Prisma, “PostgreSQL Database Connector Documentation,” 2026.

[5] Auth.js, “NextAuth.js Documentation,” 2026.